TRACKPOINT SUPPORT TOOL

CameraManager

System Documentation

Purpose: A practical guide for installing CameraManager, understanding its role in the TrackPoint workflow, and using the main camera, calibration, socket, and visualization tools.

This document is based on the current CameraManager project and updates the older 2019 user and technical guides for the modern Qt 6 / CMake version. If more detailed explanations are needed, the previous CameraManager user and technical guides are still available in the documentation folder of the project.

Contents

1. What Is CameraManager?

CameraManager is a graphical application used around the TrackPoint motion capture workflow. It helps users configure FLIR/Teledyne cameras, preview camera images, edit TrackPoint configuration files, inspect calibration results, and visualize reconstructed 3D coordinates.

It should not be understood as a simple video recorder. Its main role is to make the TrackPoint workflow easier to prepare, check, and inspect.

In practice, CameraManager can be used to:

- Detect and configure Spinnaker-compatible cameras.
- Display live images from one or more cameras.
- Check marker visibility before acquisition.
- Preview TrackPoint marker detection on camera images.
- Edit TrackPoint option/configuration files.
- Open and inspect `calibration_summary` files.
- Open image groups produced or used by TrackPoint.
- Open `socket` files containing reconstructed 3D coordinates.
- Display reconstructed points and cameras in a 3D viewer.
- Help validate trigger configuration before synchronized acquisition.

The final output of the complete workflow is usually not produced by CameraManager alone. TrackPoint is responsible for calibration, marker detection, and 3D reconstruction. CameraManager provides the interface used to prepare, launch, inspect, and understand those steps.

2. Installation Overview

CameraManager is now built with Qt 6 and CMake.

Required software:

- Qt 6.
- CMake 3.21 or newer.
- A C++17 compiler.
- OpenGL and GLU support.
- Teledyne FLIR Spinnaker SDK.
- Git, if the project is cloned from a repository.

> **Important:** Spinnaker is mandatory. The project cannot be linked correctly without the Spinnaker library.

The Spinnaker architecture must match the computer architecture. For example, an x86_64 SDK cannot be used on an aarch64 Linux system.

3. Recommended Project Layout

The repository contains:

```
CameraManager/
  CameraManager/              Runtime resources copied next to the executable
    img/                      Application icons
    props/                    Default settings and wizard parameter definitions
  include/                    Bundled legacy headers
  lib/                        Bundled legacy libraries, mainly for old setups
  src/                        C++/Qt source code
  src/3DProject/              3D visualization and TrackPoint-related tools
  CMakeLists.txt              Main CMake configuration
  documentation/              User and technical documentation
```

Runtime resources: The inner `CameraManager/` folder is important at runtime. It contains icons and settings files used by the application. The current CMake configuration copies this folder next to the executable after the build.

The old project required manually copying this folder to the build directory. With the current CMake setup, this should normally happen automatically.

4. Windows Installation

4.1 Install Qt

Install Qt 6 with Qt Creator from the Qt online installer.

Select:

- Qt 6.x for Desktop.
- Qt Creator.
- A working Desktop compiler kit.

If Qt Creator already detects a working kit and the project builds, no additional Microsoft installation is needed.

4.2 Compiler Notes

CameraManager needs a C++ compiler, but the exact setup depends on the Qt installation.

In many cases, installing Qt with a Desktop kit is enough. If Qt Creator does not detect any working compiler, install a compatible compiler toolchain and select it in the Qt Creator kit settings.

Microsoft Visual Studio Build Tools can be used for an MSVC-based setup, but they should not be considered mandatory if another Qt kit already builds the project correctly.

4.3 Install Spinnaker

Install the Windows version of the Teledyne FLIR Spinnaker SDK.

During installation, include application development components so that headers, libraries, and runtime files are installed.

Typical path:

```
C:\Program Files\Teledyne\Spinnaker
```

After installation, check that the SDK contains:

- Header files such as `Spinnaker.h` or `System.h`.
- Library files such as `Spinnaker_v140.lib`, `Spinnaker_v143.lib`, or similar.
- Runtime DLLs.

4.4 Configure in Qt Creator

Open:

```
CMakeLists.txt
```

Then:

1. Select a Qt 6 Desktop kit.
2. Configure CMake.
3. Build the `camera_manager` target.

If Spinnaker is not found automatically, add this CMake variable:

```
SPINNAKER_ROOT=C:/Program Files/Teledyne/Spinnaker
```

Use the real installation path if it is different.

4.5 Run

Run the target from Qt Creator.

If the application starts but cannot find Spinnaker DLLs, verify that the runtime DLLs are available either:

- In the executable folder.
- In the system PATH.
- In the Spinnaker installation folder copied or referenced by CMake.

5. macOS Installation

5.1 Install Qt

Install Qt 6 and Qt Creator from the Qt online installer.

Select the Qt 6 macOS desktop kit.

If Qt Creator already detects a working macOS kit and the project builds, no extra compiler installation is needed.

If no compiler is detected, install Apple's command line tools:

```
xcode-select --install
```

5.2 Install Spinnaker

Install the macOS Spinnaker SDK from Teledyne FLIR.

Common installation locations include:

```
/usr/local/include
/usr/local/lib
/opt/homebrew/include
/opt/homebrew/lib
```

Verify installation with:

```
find /usr/local /opt/homebrew -name 'libSpinnaker*.dylib' 2>/dev/null
```

5.3 Configure and Build

Open `CMakeLists.txt` in Qt Creator and select the Qt 6 macOS kit.

If Spinnaker is not found automatically, set:

```
SPINNAKER_ROOT=/usr/local
```

Use the actual SDK root if it differs.

Build and run the `camera_manager` target.

6. Linux Installation

6.1 Install Build Tools

On Ubuntu/Debian-based systems:

```
sudo apt update
sudo apt install build-essential cmake git
```

Install Qt 6 either with the Qt online installer or system packages.

Common packages:

```
sudo apt install qt6-base-dev qt6-base-dev-tools libgl1-mesa-dev libglu1-mesa-dev
```

6.2 Check Architecture

Before installing Spinnaker, check the computer architecture:

```
uname -m
```

Common results:

- x86_64: Intel/AMD 64-bit.
- aarch64: ARM64.

Download the Spinnaker SDK matching this architecture.

6.3 Install Spinnaker

Install the Linux Spinnaker SDK from Teledyne FLIR.

After installation, verify that the library and headers exist:

```
find /opt /usr/local -name 'libSpinnaker.so*' 2>/dev/null
find /opt /usr/local -name 'Spinnaker.h' 2>/dev/null
find /opt /usr/local -name 'System.h' 2>/dev/null
```

Typical installation folders:

```
/opt/spinnaker
/usr/local
```

If nothing is found, Spinnaker is not installed or is installed in a non-standard location.

6.4 Configure in Qt Creator

Open `CMakeLists.txt` in Qt Creator.

Select a Qt 6 GCC kit.

If Spinnaker is installed in `/opt/spinnaker`, add:

```
SPINNAKER_ROOT=/opt/spinnaker
```

Reconfigure CMake after changing this value. If Spinnaker was installed after the first CMake configuration, Qt Creator may need a fresh CMake configure step or a clean build folder.

6.5 Build from Terminal

Example:

```
cmake -S . -B build-linux -DCMAKE_BUILD_TYPE=Debug -DSPINNAKER_ROOT=/opt/spinnaker
cmake --build build-linux --target camera_manager -j
```

Run:

```
./build-linux/bin/camera_manager
```

6.6 USB Memory Setting for Cameras

For high bandwidth USB cameras, Linux may require increasing USBFS memory.

Check current value:

```
cat /sys/module/usbcore/parameters/usbfs_memory_mb
```

Temporary change:

```
sudo sh -c 'echo 1000 > /sys/module/usbcore/parameters/usbfs_memory_mb'
```

Persistent change can be done through the GRUB configuration by adding:

```
usbcore.usbfs_memory_mb=1000
```

Then update GRUB and reboot. This should be validated by the lab administrator before being applied on shared machines.

7. CMake Configuration

The current project is configured through CMake, not qmake.

Important CMake behavior:

- Requires Qt 6 components: Core, Gui, Widgets, Network, OpenGL, OpenGLWidgets.
- Requires OpenGL.
- Searches for Spinnaker headers and libraries.
- Supports `SPINNAKER_ROOT` to point to a custom SDK location.
- Copies Spinnaker runtime libraries when possible.
- Copies the `CameraManager/` runtime resource folder next to the executable.

Useful variables:

```
SPINNAKER_ROOT
CAMERA_MANAGER_USE_SYSTEM_SPINNAKER_HEADERS
```

On macOS, system Spinnaker headers are used by default. On other platforms, the project may use bundled headers unless a system SDK is configured.

8. First Launch Checklist

Before launching CameraManager for a lab session:

- Connect the cameras.
- Verify that cameras are visible in SpinView.
- Close SpinView if it is using the cameras.
- Launch CameraManager.
- Check that the cameras appear in the camera tree.
- Open the live views needed for the setup.
- Verify that camera serial numbers match the physical camera setup.
- Load the TrackPoint project folder if needed.

If the cameras are not visible in SpinView, CameraManager will probably not detect them either.

9. Main Interface

CameraManager is organized around four main interface areas.

9.1 Menu Bar

The menu bar provides global actions such as opening projects, managing windows, hiding interface panels, or opening supported files.

9.2 Toolbar

The toolbar provides camera and visualization actions.

Typical actions include:

- Start or stop live view.
- Update images manually.
- Take a picture / capture one image.
- Toggle crosshair display.
- Toggle coordinate display.
- Enable mosaic view.
- Remove open windows.
- Change preview quality.
- Toggle color mode when supported.

Internally, these actions are handled mainly by `MainWindow`, which dispatches them to the active camera manager or to the open subwindows.

9.3 Left Menu

The left menu contains the camera tab and the project tab.

The camera tab is used to:

- Select the active camera backend.
- View detected cameras.
- Enable or disable camera display windows.
- Rename or reset camera names.
- Create camera groups.
- View and modify camera parameters.

The project tab is used to:

- Load a TrackPoint project folder.
- Browse project files.
- Open supported files in the workspace.

9.4 Workspace Area

The central workspace contains subwindows.

Depending on what the user opens, it can contain:

- Camera live views.
- Option file editor windows.
- Calibration summary viewers.
- Image viewers.

- Socket file viewers.
- 3D visualization windows.

10. Camera Interface

10.1 Camera Detection

Camera detection is handled through the selected camera manager. In the current maintained version, Spinnaker is the relevant camera backend.

The Spinnaker backend is implemented mainly by:

```
src/spincameramanager.h
src/spincameramanager.cpp
src/spincamera.h
src/spincamera.cpp
```

`SpinCameraManager` detects cameras through the Spinnaker system API and creates `SpinCamera` objects. `SpinCamera` then handles initialization, acquisition, image conversion, camera properties, and trigger configuration.

10.2 Camera Naming and Selection

Each camera appears in the camera tree and can be opened in a camera display subwindow.

The current interface displays camera windows using a title similar to:

```
Camera 0 - SERIAL_NUMBER
```

This helps match a display window with a physical camera.

Clicking a camera display selects the corresponding camera in the camera tree. This makes it easier to adjust the parameters of the camera currently being viewed.

10.3 Camera Groups

Camera groups can be created from the camera tree context menu.

Groups are useful when many cameras are connected and the operator wants to organize them by physical location, calibration combination, or acquisition role.

10.4 Camera Properties

When a camera is selected, its properties are displayed in the left panel.

Common Spinnaker properties include:

- Brightness.
- Gain.
- Exposure.
- Gamma.
- Shutter.
- Frame rate.
- Trigger enabled.
- Software trigger.

Each property can have a name, an automatic/manual control, a writable value, a read value, and a slider.

The availability of properties depends on the camera model and on Spinnaker node permissions. Some properties are not writable while the camera is acquiring images.

Camera property definitions and default settings are stored in:

```
CameraManager/props/defaultCameraSettings.ini
CameraManager/props/spinCameraSettings.ini
CameraManager/props/quickfile_camerasettings.ini
```

11. Live View and Image Display

Live view is used to check framing, focus, exposure, marker visibility, and camera behavior.

The image display is OpenGL-based. It can show:

- The current camera image.
- Crosshair and image coordinates.
- TrackPoint preview overlays.
- Marker labels when enabled.
- Filtered images when TrackPoint filtered preview is enabled.

The relevant display classes are:

```
src/videoopenglwidget.h
src/videoopenglwidget.cpp
src/imageopenglwidget.h
src/imageopenglwidget.cpp
src/texture.h
src/texture.cpp
```

`VideoOpenGLWidget` receives camera frames. `ImageOpenGLWidget` handles texture upload, drawing, mouse coordinates, crosshair display, TrackPoint overlays, and display-level image transformations.

The live view is mainly a setup and monitoring feature. The final acquisition pipeline may use external trigger logic and TrackPoint processing rather than the live preview itself.

12. TrackPoint Preview

TrackPoint preview performs marker detection directly inside CameraManager for visual verification.

It is controlled by the TrackPoint settings tab and these settings files:

```
CameraManager/props/defaultTrackPointSettings.ini
CameraManager/props/quickfile_trackpoint.ini
```

Main parameters:

- `Threshold`: brightness threshold used to find candidate marker pixels.
- `SubWindow`: local area used around detected candidates.
- `Min Point Size`: minimum accepted point size.
- `Max Point Size`: maximum accepted point size.
- `Minimal Separation`: minimum distance between two accepted points.
- `TrackPoint Preview`: displays detected points on top of the image.
- `Filtered Image Preview`: displays the thresholded/filtered image.
- `Show Coordinate Labels`: shows coordinate labels beside points.
- `Show Min Separation Circle`: shows the separation radius around points.
- `Remove Duplicates`: removes duplicated detections.

Recommended usage:

- Enable preview first on one camera.
- Adjust threshold until markers are detected reliably.
- Check false positives and missed points.
- Enable additional cameras only after one camera is correctly configured.
- Disable preview when it is no longer needed to reduce CPU usage.

TrackPoint preview is implemented in CameraManager, not by Spinnaker. Spinnaker provides camera access and frames; the marker preview processing is done by the application.

The relevant implementation files are:

```
src/imagedetect.h
src/imagedetect.cpp
src/trackpointproperty.h
src/imageopenglwidget.h
src/imageopenglwidget.cpp
```

13. TrackPoint Project Files

CameraManager can load a TrackPoint project folder and open the files used by the TrackPoint workflow.

Historically, the selected folder should contain `trackpoint` in its name. This convention helps the application identify the project root.

After loading, CameraManager recursively displays supported project files in the project tree.

Supported file types include:

- Option/configuration files.
- `calibration_summary` files.
- Combination calibration files.
- Socket coordinate files.
- Grupper images.
- Text files and executables when relevant.

14. Option File Viewer

The option file contains parameters used by TrackPoint.

CameraManager provides two ways to view and edit it:

- Raw text mode.
- Wizard mode.

14.1 Raw Text Mode

Raw text mode shows the file exactly as stored on disk.

Use it when:

- The wizard does not recognize the file structure.
- A parameter is not exposed by the wizard.
- Exact manual editing is needed.

Editing is normally disabled by default to avoid accidental changes. Enable editing from the context menu, then save manually.

There is no autosave. If the file is edited and not saved, changes can be lost.

14.2 Wizard Mode

Wizard mode displays selected parameters as graphical fields.

The list of parameters shown in the wizard is configured in:

```
CameraManager/props/parameterList.txt
```

This makes it possible to add or remove visible fields without rewriting the whole viewer, as long as the option file format remains compatible.

The relevant implementation files are:

```
src/configfileviewerwidget.h
src/configfileviewerwidget.cpp
src/configfilereader.h
src/configfilereader.cpp
```

14.3 Launching TrackPoint

The current code contains support for launching a TrackPoint executable from the option file viewer.

The default executable path is defined by constants in the code and may need to be adapted to the lab computer.

To complete after lab validation:

- Confirm the expected TrackPoint executable location.
- Confirm whether CameraManager should launch TrackPoint directly in the final workflow.
- Confirm which option file should be passed to the executable.

15. Calibration Viewer

The `calibration_summary` file is one of the main outputs of a TrackPoint calibration run.

It normally includes:

- Camera groups or selected camera combinations.
- Paths to image groups or related files.
- Camera identifiers and serial numbers.
- Quality indicators for camera combinations.
- Failed combinations, often marked with `NO CONVERGENCE`.

CameraManager can display this file in text view and table view.

15.1 Text View

The text view displays the calibration file directly.

Context actions can include:

- Go to top.
- Enable or disable file editing.
- Save the file.

Left-click actions on combination lines can include:

- Open the detailed combination file.
- Select a combination.

- Show or hide failed combinations.
- Show or hide useless combinations.
- Sort combinations by calibration metrics.
- Apply combo sorting for paired combinations.

15.2 Combination Selection

Combination selection is used to choose which calibrated camera combinations should be used later in the TrackPoint workflow.

A combination marked as failed should not be selected for production use.

When a combination is selected, the application can mark other combinations as less useful if they reuse the same cameras. This helps the operator choose a coherent set of combinations.

15.3 Table View

The table view presents the selected combination in a more readable way.

It shows camera identifiers, serial numbers, and camera-specific calibration parameters. This is useful when the raw file contains many columns and is difficult to read manually.

The relevant implementation files are:

```
src/calibrationviewerwidget.h
src/calibrationviewerwidget.cpp
src/calibrationfile.h
src/calibrationfile.cpp
src/calibrationfileopenglwidget.h
src/calibrationfileopenglwidget.cpp
```

16. Grupper Images

Grupper images are image groups used by TrackPoint during production runs.

CameraManager can open these images and display detected points. Clicking a point can show information such as camera number, point number, and image coordinates.

The historical workflow usually stores these images in a folder named similar to:

```
images_grupper
```

The relevant implementation files are:

```
src/imageviewerwidget.h
src/imageviewerwidget.cpp
src/imagelabel.h
src/imagelabel.cpp
```

17. Socket Viewer and 3D Visualization

The socket file contains reconstructed 3D coordinates over time.

Its name often follows a timestamp pattern such as:

```
socket_YYYYMMDD_HHMMSS
```

CameraManager can display socket data as raw text, coordinate tables, or a 3D visualization.

17.1 Text View

Text view shows the raw socket file. It is useful for debugging, but not convenient for reading many coordinates over time.

17.2 Coordinate Table

The table view displays one time step per row and point coordinates in columns.

The interface can show tooltips when hovering over values, indicating which point and coordinate axis is being inspected.

17.3 3D View

The 3D view can display:

- Reconstructed points.
- Lines between points.
- Previous positions for motion context.
- Camera positions from calibration data.
- Camera legs.
- Camera field of view.
- Coordinate axes.
- Floor/grid reference.
- Orthographic or perspective-style views depending on options.

The 3D view is intended for qualitative verification of TrackPoint output, not for replacing numerical analysis.

The relevant implementation files are:

```
src/socketviewerwidget.h
src/socketviewerwidget.cpp
src/widgetgl.h
src/widgetgl.cpp
```

18. Swapping Corrector

The current code includes a Swapping Corrector integration from the socket viewer.

Its purpose is to help inspect or correct point identity swaps in reconstructed data.

The related files are stored in:

```
src/3DProject/
```

Important files include:

```
src/3DProject/swappingcorrectorprogram.h
src/3DProject/swappingcorrectorprogram.cpp
src/3DProject/programwindow.h
src/3DProject/programwindow.cpp
src/3DProject/data.h
src/3DProject/data.cpp
```

To complete after lab validation:

- Confirm the expected user workflow.
- Confirm which files are read and written by this tool.
- Add an example before/after correction.

19. Trigger Mode

Trigger mode controls when a camera acquires images.

19.1 Continuous Acquisition

When trigger mode is disabled, the camera acquires continuously according to its acquisition settings, exposure, and frame rate limits.

This is the simplest mode for live preview and basic camera setup.

19.2 Software Trigger

When trigger mode is enabled and the trigger source is software, the camera waits for a software command.

CameraManager can send a software trigger command when appropriate.

Software trigger is useful for testing, but it is generally not the best solution for tightly synchronized multi-camera acquisition.

19.3 Hardware Trigger

When trigger mode is enabled and the trigger source is a hardware line, the camera waits for an electrical signal on the configured input line.

This is usually the preferred method for synchronized acquisition because the same signal can be distributed to several cameras.

For hardware trigger setup, validate the signal in SpinView before using CameraManager.

Check:

- `TriggerMode`.
- `TriggerSource`.
- Selected line, such as `Line0`.
- `LineStatus` while the external device sends pulses.
- Electrical compatibility between trigger generator and camera input.

> **Trigger troubleshooting:** If `LineStatus` does not change in SpinView, the problem is wiring, voltage level, camera line selection, or camera input configuration rather than CameraManager.

20. Technical Architecture Summary

This section summarizes the main code structure for maintenance.

20.1 Entry Point and Main Window

The application starts in:

```
src/main.cpp
```

It creates the Qt application and opens `MainWindow`.

`MainWindow` coordinates:

- Menu bar and toolbar actions.
- Camera tree events.
- Project tree events.
- TrackPoint settings UI.
- Camera property updates.
- Camera detection.
- Opening project files in the correct viewer.

Important files:

```
src/mainwindow.h
src/mainwindow.cpp
src/ui_mainwindow.h
src/ui_mainwindow.cpp
src/menubar.h
src/menubar.cpp
```

20.2 Camera Abstraction

Core classes:

```
src/abstractcamera.h
src/abstractcamera.cpp
src/abstractcameramanager.h
src/abstractcameramanager.cpp
```

`AbstractCamera` represents one camera. `AbstractCameraManager` represents a camera backend and handles camera detection, camera tree data, live view windows, and property widgets.

20.3 Spinnaker Backend

Important files:

```
src/spincamera.h
src/spincamera.cpp
src/spincameramanager.h
src/spincameramanager.cpp
src/spincameraproperty.h
src/spincameraproperty.cpp
src/systemmanager.h
src/systemmanager.cpp
```

This backend handles camera discovery, image acquisition, Spinnaker node reads/writes, frame rate configuration, and trigger configuration.

20.4 File Viewers

File viewers are separated by file type:

- Configuration files: `ConfigFileViewerWidget` and `ConfigFileReader`.
- Calibration files: `CalibrationViewerWidget` and `CalibrationFile`.
- Image files: `ImageViewerWidget` and `ImageLabel`.
- Socket files: `SocketViewerWidget` and `WidgetGL`.

This separation makes the code easier to maintain because each file type has its own viewer logic.

21. Troubleshooting

21.1 Spinnaker Not Found During Build

Symptoms can include:

```
collect2: error: ld returned 1 exit status
```

or CMake cannot find Spinnaker.

Check:

```
uname -m
find /opt /usr/local -name 'libSpinnaker.so*' 2>/dev/null
```

Fix:

- Install Spinnaker.
- Install the correct architecture package.
- Set `SPINNAKER_ROOT`.
- Reconfigure CMake.

21.2 Camera Not Detected

Check:

- Camera power and USB connection.
- USB3 cable and port.
- Camera visibility in SpinView.
- Whether another application already controls the camera.
- Spinnaker installation and drivers.

21.3 Application Starts but Icons or Settings Are Missing

Check that this folder exists next to the executable:

```
CameraManager/
```

If missing, rebuild with CMake or copy it manually as a temporary workaround.

21.4 Live View Is Slow

Possible causes:

- Too many cameras streaming at high frame rate.
- TrackPoint preview enabled on several cameras.

- CPU saturation.
- USB bandwidth limitation.
- Exposure time too high for requested frame rate.
- Preview quality setting too expensive.

Try:

- Disable TrackPoint preview.
- Test one camera at a time.
- Reduce frame rate.
- Reduce exposure time.
- Avoid USB hubs.
- Check CPU and memory usage.

21.5 Trigger Does Not Work

Check trigger behavior in SpinView before CameraManager.

> **Trigger troubleshooting:** If `LineStatus` does not change when the external device sends pulses, inspect wiring and camera input configuration.

Potential causes:

- Wrong GPIO pin or trigger generator output.
- Missing common ground.
- Wrong camera connector pin.
- Wrong trigger source line.
- Voltage level incompatible with camera input.
- Camera input requires a specific circuit or opto-isolated wiring.